

밈 주식 Contrarian 효과 예측을 위한 머신러닝 모델링 매뉴얼

1. 데이터 수집 및 전처리

1.1 Reddit 소셜미디어 데이터 수집: 밈 주식에 대한 Reddit 게시물 데이터를 수집하기 위해 공식 Reddit API나 Pushshift와 같은 아카이브 API를 활용한다. 예를 들어 *r/wallstreetbets* 커뮤니티에서 2021년 1월부터 2023년 12월까지 GME, AMC, BB 등의 티커가 언급된 모든 게시물(및 댓글)을 크롤링한다 ^①. Python에서는 `praw` 라이브러리나 `Pushshift API`를 사용할 수 있다. 각 게시물의 **작성 시간(timestamp)**, **작성자**, **제목/본문 내용(text)**, **업보트 수(score)**, **댓글 수** 등의 메타데이터를 확보하고, 원하는 경우 댓글도 별도로 수집한다. 수집된 데이터는 CSV 혹은 DB에 저장하고, 추후 일자별 통계로 집계할 수 있도록 한다. 불필요한 정보(예: 삭제된 계정의 포스트 등)는 필터링하고, **특정 티커 언급 여부**를 기준으로 데이터를 분리하여 종목별로 볼 수 있게 정리한다. 예를 들어 게시물 내용에 "GME"가 포함되어 있으면 GME 관련 포스트로 분류한다.

1.2 주가 가격 데이터 수집: 주가 데이터는 Yahoo Finance API 등을 이용해 동일 기간(2021~2023)의 **일별 가격** 정보를 가져온다 ^①. Python의 `yfinance` 라이브러리를 사용하면 손쉽게 **시가, 고가, 저가, 종가, 거래량** 등의 데이터를 다운로드할 수 있다. 본 연구에서는 일별 **종가와 거래량**을 주로 활용하며, 필요시 전일 종가 대비 수익률을 계산한다. 수집한 가격 데이터는 Reddit 데이터와 결합하기 위해 **거래일 기준의 날짜 열**을 포함하도록 한다. 종가 기준 **일간 수익률**은 다음과 같이 계산할 수 있다:

$$\text{Return}_t = \frac{P_t - P_{t-1}}{P_{t-1}}$$

여기서 P_t 는 t일 종가이다 ^②. Python 예시: `df['return'] = df['Close'].pct_change()`. 이후 모델의 예측 목표(label)로 **다음 거래일의 수익률**을 사용하기 위해, 수익률 열을 한 칸 쉬프트하여 `df['next_return'] = df['return'].shift(-1)` 처럼 **t일의 레이블을 t+1일 수익률**로 정의한다. 이렇게 하면 Reddit 데이터가 **당일(t)의 관심 급증이 다음날(t+1) 수익률에 영향을 준다**는 가설에 부합하도록 라벨링된다 ^③.

1.3 데이터 병합 및 시간 정렬: Reddit 데이터와 주가 데이터를 일자별로 합친다. 우선 Reddit 게시물의 타임스탬프를 **거래 날짜**로 매핑한다. 일반적으로 **전일 장 마감 후 ~ 당일 장 마감 시점까지**의 게시글을 모아 해당 거래일의 지표로 삼고, 그에 대한 **다음 거래일**의 주가 움직임을 예측하도록 설계한다. 구체적으로, 1월 5일 거래일의 Reddit 특성들은 1월 6일의 주가 수익률 예측에 사용된다 (미래 정보 유출을 방지하기 위함). 주말이나 공휴일 동안의 게시글은 다음 영업일에 합쳐 반영할 수 있다. 예를 들어 토~일요일의 게시글은 월요일 데이터에 포함시키고 `is_monday` 특성으로 주말 효과를 보정한다. Pandas에서는 다음과 같이 할 수 있다:

```
# Reddit 데이터프레임: df_reddit, 주가 데이터프레임: df_price
# Reddit timestamps를 날짜로 변환하고 날짜별로 mention 수 합산 등 집계
df_reddit['date'] = pd.to_datetime(df_reddit['timestamp']).dt.date
daily_reddit = df_reddit.groupby(['date', 'ticker']).agg({
    'post_id': 'count', 'score': 'sum', 'comment_count': 'sum', ...})
```

```

daily_reddit = daily_reddit.rename(columns={'post_id': 'num_posts'})
# 주가 데이터와 병합
data = pd.merge(df_price, daily_reddit, how='left',
on=['date', 'ticker']).fillna(0)
# 다음날 수익률 레이블 생성
data['return'] = data.groupby('ticker')['Close'].pct_change()
data['next_return'] = data.groupby('ticker')['return'].shift(-1)

```

위와 같이 **티커별 그룹핑**을 통해 데이터를 구성하면, 여러 종목의 데이터를 하나로 통합하여 모델을 학습시킬 수 있다. 이 때 **티커 정보를 특성으로 포함**하거나, 종목별로 별도 모델을 만드는 두 가지 접근이 가능하다. 데이터 병합 후에는 **결측치 처리**와 **정규화**를 수행한다. 예를 들어 Reddit 활동이 없던 날의 해당 특성은 0으로 채우거나, 극단치(이상치)는 윈저라이징(Winsorizing)이나 로그 변환(예: `log_mentions`)으로 처리한다 ⁴. 또한 **정규화**는 각 특성의 스케일 차이를 줄여주므로 Min-Max 또는 Z-스코어 정규화를 적용할 수 있다 (특히 딥러닝 모델 사용 시). 이로써 최종적으로 **일자별 (또는 종목-일자별) 하나의 행으로 구성된 학습용 데이터셋**이 완성된다.

2. 피처 엔지니어링 (Feature Engineering)

밈 주식의 *Contrarian effect*를 포착하기 위해서는 가격 데이터뿐만 아니라 **소셜미디어 기반 특성**들을 정교하게 설계해야 한다. 기존 연구에서는 22개의 특성을 사용하였는데, 과거 수익률/변동성, 기술적 지표, Reddit 활동 지표, 시장 감정, 요일 효과 등이 포함되었다 ⁵ ⁶. 여기서는 이러한 기본 구조를 확장하여 **텍스트 내용 분석에 기반한 특징들과 레짐(Regime) 변수** 등을 추가로 구축하는 방법을 다룬다. 각 피처는 가능하면 구체적인 수식이나 코드로 정의하고, 연구자가 동일한 과정을 재현할 수 있도록 한다.

2.1 텍스트 기반 특성: 확신도, 참여도, 감정, 토픽

- **확신도(Confidence) 지표**: Reddit 게시물에서 작성자가 얼마나 **확신에 찬 어조**를 사용하는지를 계량화한다. 기존 연구에서는 게시물 내 **강한 확인적 표현의 빈도**를 측정하여 *confidence_score*를 부여했고, 이 값이 높을수록 다음날 주가수익률이 유의미하게 하락하는 **역상관(-0.205)** 관계를 발견하였다 ⁷. 이를 모델에 활용하기 위해, 예를 들어 **“guaranteed”, “definitely”, “확실하다”** 등의 확신 단어 리스트를 구성하고, 각 게시물마다 이러한 단어가 등장하는 횟수를 세어 **포스트별 확신 점수**를 계산한다. 반대로 “maybe”, “가능성”, “추측컨대” 등 **불확실성을 나타내는 표현**이 많으면 점수를 낮게 줄 수 있다. 각 일자에 대해 모든 관련 게시글의 확신 점수를 **평균 또는 합계**하여 일별 *Confidence* 지표로 사용한다. 아래는 간단한 예시 코드이다:

```

confident_words = ["guaranteed", "definitely", "surely", "확실", "반드시"]
hesitant_words = ["maybe", "perhaps", "아마", "추측", "불확실"]
def confidence_score(text):
    text = text.lower()
    score_conf = sum(word in text for word in confident_words)
    score_hes = sum(word in text for word in hesitant_words)
    return score_conf - score_hes

df_reddit['conf_score'] = df_reddit['text'].apply(confidence_score)
daily_confidence = df_reddit.groupby('date')['conf_score'].mean()

```

이렇게 얻은 *confidence_score* 특성은 투자자들의 **과잉확신 편향**을 포착하는 핵심 변수로, 모델이 **확신도가 높을수록 오히려 음의 수익률을 예측**하도록 학습하는데 도움을 줄 것이다 ⁸ .

- **참여도(Engagement) 지표:** 커뮤니티 내 **관심도와 반응** 정도를 나타내는 특성들이다. 예를 들어 **일별 게시글 수** (mention count) 및 **댓글 수 합계, 업보트(upvote) 합계** 등을 들 수 있다. 기존 연구의 핵심 변수인 **Reddit Surprise (RS)**는 실제 언급량이 예상치를 초과한 정도를 나타내는 지표로 정의되었는데 ⁹ , 이를 재현하려면 **예상 언급량**을 계산해야 한다. 예상 언급량은 과거 평균이나 추세로부터 예측된 값으로 볼 수 있는데, 예를 들어 직전 7일 간의 **이동평균 또는 이동평균선 추정치**를 사용하거나, 요일/월 효과를 반영한 **기대값 모델**(예: 주중 평균, 월별 평균)을 수립할 수 있다. 한 방법으로, 7일 이동평균을 기대치로 두고 RS를 계산하면:

$$RS_t = \frac{\text{ActualMentions}_t - \text{MA}_{7,t}}{\text{MA}_{7,t}}.$$

여기서 ActualMentions_t 는 t일의 실제 게시글 수, $\text{MA}_{7,t}$ 는 t일 기준 7일 이동평균이다. RS 값이 **높을수록** 해당 일에 **이례적으로 많은 관심(spike)**이 집중되었음을 의미한다. 예를 들어 $RS > 1$ 이면 실제 언급량이 평소의 두 배 이상이다. 이외에도 **참여도**를 나타낼 수 있는 특성으로: - 평균 게시글 길이 또는 감정이 담긴 댓글 비율: 커뮤니티 토론의 **열기**를 반영 - 고유 작성자 수: 해당 종목에 언급하는 **사람들의 지변** (많은 사람이 언급하면 광범위한 관심) - 상위 n% 인기 게시글 여부: 예를 들어 당일 **가장 업보트를 많이 받은 글**이 해당 종목에 관한 것인지를 이진 변수로 표시 (이는 폭발적인 이슈 여부를 나타냄)

등을 추가로 고려할 수 있다. 이러한 참여도 지표들은 **소셜 미디어의 열기**가 주가에 미치는 영향을 정량화하는데 기여한다.

- **감정(Sentiment) 지표:** 게시글의 **감정적 톤(긍정/부정)**을 측정하는 특성이다. 일반적으로 감정 분석 도구를 활용하여 각 글의 **폴라리티 점수**(예: -1=매우 부정, +1=매우 긍정)를 산출하고, 이를 일별로 평균내어 일일 평균 감정 점수로 사용한다. 금융 도메인에 특화된 **FinBERT** 모델이나 경량 도구인 **VADER**를 사용할 수 있다. 예를 들어 VADER를 적용하면 Python에서:

```
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
analyzer = SentimentIntensityAnalyzer()
df_reddit['sentiment'] = df_reddit['text'].apply(lambda x:
analyzer.polarity_scores(x)['compound'])
daily_sentiment = df_reddit.groupby('date')['sentiment'].mean()
```

이렇게 구한 감정 점수 (예: +0.2는 다소 긍정, -0.3은 다소 부정)는 투자자들의 **평균 심리 상태**를 반영한다. 기존 가설과 달리 밈 주식에서는 **과도한 긍정 심리**가 오히려 주가에 부정적 영향을 줄 가능성이 있으므로, 모델이 이러한 역관계를 학습할 수 있도록 피쳐로 포함한다. (필요 시 긍/부정 비율, 부정어 사용빈도 등의 세부 지표도 고려 가능하다.)

- **토픽(Topic) 지표:** 수만 건의 게시글에 내재한 **대화 주제의 유형**을 정량화하여, 단순 언급량 외에 **내용상의 차이**를 반영한다. 예를 들어 LDA(Latent Dirichlet Allocation)와 같은 토픽 모델링을 적용해 5~10개의 주요 토픽을 추출할 수 있다. 밈 주식의 경우 예상되는 토픽으로는 **기술적 분석, 펀더멘털/실적 논의, 단순 밈/유행어**(“to the moon”), **포트폴리오 공유(손익 인증), 시장 뉴스 공유** 등이 있을 수 있다. 토픽 모델 적용 후 각 게시글에 대해 **토픽 분포**를 얻고, 이를 일자별로 집계한다. 간단하게는 **주요 토픽의 게시글 수**를 일별 피쳐로 만들거나, **특정 토픽 비중**(해당 토픽 글 수 / 전체 글 수)을 구해 사용할 수 있다. 예시로 5개 토픽으로 LDA를 훈련했다면, 일자별로 토픽별 게시글 수를 features로 갖는 5차원 벡터를 생성한다. 만약 “밈/유머” 토픽 비중이 높아지는 날에는 투자 논의가 아니라 놀이에 치중된 날로 볼 수 있고, 이러한 날의 주가 영향이 다를 수 있다.

이처럼 텍스트 기반 특성들은 **정성적 데이터**를 **정량화**하여 모델에 활용하는 것으로, 기존 연구에서 확인된 **Behavioral Paradox** (확신 표현 ↑ → 성과 ↓) 같은 현상을 모델이 학습하도록 돕는다 ⁸. 나아가 이러한 특성들을 다른 정량 지표와 결합함으로써 **Contrarian 효과의 원인**을 더 정확히 포착할 수 있다.

2.2 Reddit 활동 모멘텀 및 변동 특성

- **Reddit 모멘텀 지표:** 소셜미디어상의 관심이 **최근에 얼마나 증가 또는 감소**했는지를 나타내는 동적 지표이다. 예를 들어 3일 모멘텀은 현재 언급량이 3일 전 대비 얼마나 증가했는지를 나타낼 수 있다. 수식으로

$$\text{RedditMomentum}_{3,t} = \text{Mentions}_t - \text{Mentions}_{t-3},$$

$$\text{RedditMomentum}_{7,t} = \text{Mentions}_t - \text{Mentions}_{t-7},$$

와 같이 정의할 수 있다 (또는 비율 변화로 $(\text{Mentions}_t / \text{Mentions}_{t-3}) - 1$ 형태도 가능). 실제 논문에서도 *reddit_momentum_3*, *reddit_momentum_7* 특성을 사용하였으며 ¹⁰, 이는 단순 언급량보다 **증가 속도**에 초점을 맞춘 값이다. 이러한 모멘텀 특성은 **관심 증폭/감퇴의 속도**가 주가에 미치는 영향을 학습하기에 유용하다. 예를 들어 언급량이 **급증**하는 추세인지, **둔화**되는 추세인지에 따라 다음 날 주가 움직임이 다를 수 있다.

- **지수평활 이동평균 (EMA) 특성:** 언급량 시계열에 **지수 가중치**를 두어 단기 추세를 포착한 값이다. 논문에서는 *reddit_ema_3*, *reddit_ema_5*, *reddit_ema_10*을 사용했는데 ⁴, 예를 들어 *reddit_ema_3*는 지난 3일간 언급량의 **EMA(지수이동평균)**이다. EMA는 최근 데이터에 더 높은 가중치를 부여하므로,

$$\text{EMA}_{k,t} = \alpha \cdot \text{Mentions}_t + (1 - \alpha)\text{EMA}_{k,t-1},$$

여기서 $\alpha = \frac{2}{k+1}$. 즉, 3일 EMA의 $\alpha = 0.5$, 5일 EMA는 $\alpha \approx 0.33$. EMA 특성들은 노이즈를 줄이면서 **단기 추세 레벨**을 반영하여, 갑작스런 급등치보다는 **최근 며칠간의 평균적 관심 수준**을 모델에 제공한다. 이러한 값이 높다는 것은 **지속적으로 관심이 높았다**는 의미이고, 낮다가 갑자기 언급량이 급증하면 EMA 대비 실제 Mentions가 크게 상승하여 RS도 높아지게 된다.

- **Reddit 변동성/불균형 지표:** 소셜미디어 내 **논쟁의 열기**나 **의견 불일치** 정도를 나타내는 특성도 고려할 수 있다. 예를 들어 댓글 대비 게시물 비율, 감정 극성(강한 긍정 + 강한 부정의 비율), 특정 밈/이슈의 집중도(예: 한 두 개 게시물에 댓글과 업보트가 몰리는 집중 현상) 등이 있다. 이러한 값은 커뮤니티 내 **활동 분산도**를 의미하며, 너무 국지적으로 일부 게시물에만 관심이 쏠렸다면 쏠림 현상을 나타낸다. 다만 이러한 특성은 구현이 다소 복잡하므로, 핵심적인 contrarian 신호와 직접 관련된 위의 언급량 수준/증감 위주로 우선 구성하고, 여력이 있다면 추가적으로 실험해 볼 수 있다.

2.3 기술적 지표 특성

- **가격 모멘텀/추세:** 주가 자체의 **과거 수익률**과 **추세** 관련 지표들이다. 예를 들어 논문에서 사용된 과거 1일, 3일, 5일, 10일 수익률 (*returns_1d*, *returns_3d*, ... *returns_10d*) ⁵은 각각 해당 기간 동안 주가가 얼마나 올랐는지 (혹은 내렸는지)를 나타낸다. 이것들은 **가격 추세의 단기 연속성 또는 반전** 효과를 모델이 학습하도록 도와준다. 예를 들어 연속 급등 후 조정 가능성 등.

간단히 Pandas로 계산하면: `data['ret_3d'] = data['Close'].pct_change(3)` (3일 전 대비 등락) 또는 `data['ret_3d'] = data['Close'].shift(0)/data['Close'].shift(3) - 1` 등으로 구할 수 있다. 이

와 유사하게 **누적 수익률**이나 **모멘텀 지수**(momentum indicator)도 고안할 수 있다. 예를 들어 모멘텀 지수 10 = 현재 가격이 10일 전보다 몇 % 높은지.

- **이동평균 및 괴리도**: 주가의 이동평균선과 현재 가격의 차이를 특성화한 것이다. 논문에서는 $price_ratio_sma10$, $price_ratio_sma20$ 을 사용했는데, 이는 현재 종가를 10일, 20일 단순이동평균(SMA)으로 나눈 값으로 추정된다¹¹. 예를 들어

$$PriceRatioSMA_{10,t} = \frac{P_t}{SMA_{10,t}},$$

$$PriceRatioSMA_{20,t} = \frac{P_t}{SMA_{20,t}},$$

로 정의하면 1보다 큰 경우 현재 가격이 이동평균보다 높아서 **상승 추세**임을, 1보다 작으면 **하락 추세**임을 의미한다. 이를 %로 나타내고 싶으면 100을 곱하거나 $(P_t - SMA_{10})/SMA_{10}$ 형식으로 **괴리율**을 구할 수도 있다. 이러한 추세 지표는 **평균회귀 vs 추세 지속 여부**를 판단하는 데 도움을 준다.

- **모멘텀 진동지표 (RSI 등)**: $RSI(14)$ 는 14일 기간 동안 **상승폭과 하락폭의 비율**로, 0~100 사이의 값이며 일반적으로 70 이상이면 과매수, 30 이하이면 과매도로 판단한다. 논문에서도 $RSI(14)$ 를 사용하였으며¹¹, 이는 투자자들의 **기술적 과열/침체 상태**를 포착하는 변수이다. RSI는 Pandas or TA-Lib 등을 통해 계산할 수 있다. 예를 들어 `pandas_ta` 라이브러리를 쓰면 `df['RSI_14'] = df.ta.rsi(length=14)` 식으로 쉽게 구할 수 있다. RSI 이외에도 **스토캐스틱 오실레이터**, **MACD (이동평균 수렴-확산)**의 시그널, **볼린저 밴드 폭** 등 다양한 기술적 지표를 추가로 고려할 수 있다. 다만 특성 수가 너무 많아지면 과적합 위험이 있으므로, 우선 핵심적인 것(이동평균, RSI, 변동성 등)만 포함하고, 나중에 필요에 따라 실험적으로 추가한다.

- **변동성 지표**: 주가의 **변동성(Volatility)**을 나타내는 특성으로, 일반적으로 일정 기간의 **수익률 표준편차**를 사용한 다. 논문에서는 5일, 10일, 20일 변동성을 사용했는데¹², 이는 다음과 같이 계산할 수 있다:

$$Volatility_{5,t} = \sqrt{\frac{1}{5} \sum_{i=0}^4 (r_{t-i} - \bar{r}_{5,t})^2},$$

(혹은 **표본표준편차**로 계산). Pandas로는 `data['vol_5d'] = data['return'].rolling(5).std()` 처럼 구할 수 있다. 변동성 지표는 **위험도나 불확실성**의 척도로서, 소셜미디어 관심 급등 상황에서 변동성이 커지는지 등을 모델이 학습할 수 있게 한다. 특히 가격변동성이 큰 날에는 contrarian 효과가 다르게 나타날 수 있으므로, 모델이 이를 조건부로 다룰 수 있도록 한다.

- **시장(시장전체) 요인**: 밈 주식 특성이더라도 **시장 전반의 움직임**을 무시할 수 없다. 따라서 **벤치마크 지수 수익률** (예: S&P500 또는 나스닥 지수의 일일 수익률)이나 **섹터 ETF 수익률** 등을 추가 입력으로 넣을 수 있다. 이를 통해 시장이 폭락하는 날과 급등하는 날에 대해 모델이 구분하여 예측할 수 있고, 혹은 시장과 무관하게 밈 주식만 과열된 경우를 감지할 수 있다. 이러한 시장 요인은 반드시 필요한 것은 아니지만, 성능 향상을 위해 고려할 수 있는 부가 변수이다.

요약하면, 기술적 특성들은 **가격 자체의 추세와 상태**를 나타내고, 텍스트 기반의 Reddit 특성과 상호보완적으로 모델의 입력을 구성한다. Table 1과 같이 주요 피쳐들을 정리하면 다음과 같다:

특성 그룹	예시 변수	의미
과거 수익률	returns_1d, returns_5d, returns_10d	과거 1일/5일/10일 간 누적 수익률
기술적 지표	price_ratio_sma10, rsi_14	현재 가격의 추세 위치, RSI를 통한 과매수/과매도
변동성	vol_5d, vol_20d	단기(1주) 및 중기(1달) 가격 변동성 (표준편차)
소셜 언급량	log_mentions, reddit_ema_5, RedditMomentum_7	일별 언급량 (로그), 최근 언급량의 추세 수준, 7일 전 대비 변화
텍스트 분석	confidence_score, sentiment_avg	게시글 어조 확신도, 평균 감정 점수
참여도	num_posts, num_comments, top_post_score	일별 게시글 수, 댓글 수, 최고 인기글의 업보트 수
토픽 분포	topic1_ratio, topic2_ratio, ...	주요 토픽별 게시글 비중 (예: 밈/유머 vs 정보분석)
요일/월 효과	day_of_week, is_monday, is_friday, month	요일 (0-6) 또는 특정 요일 여부, 월별 계절성

위 표의 변수들은 기존 논문에서 사용된 것과 추가 제안된 것들을 합친 예시이다. 연구 목적과 데이터 상황에 맞게 이러한 특성들을 선택적으로 사용하되, **과적합을 피하기 위해 특징 수 대비 관측치 수**를 고려해야 한다. 특성 간 다중공선성도 문제 될 수 있으므로 (예: returns_5d와 returns_10d 간 상관 높음) VIF(Variance Inflation Factor) 등을 통해 필요시 차원 축소나 제거를 검토한다 ^{13 14}.

2.4 레짐(Regime) 특성 구성

Contrarian 효과는 **시장 상태나 이벤트 상황**에 따라 강도가 달라질 수 있으므로, **레짐(regime) 변수**를 도입하여 모델이 **상황별로 다른 거동**을 할 수 있게 돕는다. 레짐 특성은 일종의 **상태 표시기**로 0/1 (또는 몇 가지 구간) 값으로 나타내는 경우가 많다:

- **소셜미디어 과열 레짐**: Reddit 언급량이 예외적으로 높은 시점을 식별하여, 그 날들을 1로 표시하는 변수이다. 예를 들어 RS 값이 일정 임계치 이상 (예: 상위 5% 수준)인 날을 *Spike* 이벤트로 정의하고 `is_reddit_spike = 1` (그 외 0)으로 한다. 실제 논문에서도 극한 *Reddit* 스파이크 상위 5% 이벤트에 대해 이벤트스터디를 수행하여, 당일 큰 음수 수익률과 이후 며칠간의 회복/추가조정 패턴을 확인하였다 ¹⁵. 이러한 날들은 일반적인 날과 다른 패턴을 보이므로, 모델이 이를 별도로 인식하도록 피처를 주면 효과적이다. 구현상으로는, 과거 데이터에서 RS의 95퍼센타일 값을 구해 threshold로 정하고, 각 일자의 RS와 비교하여 변수 생성이 가능하다.
- **시장 변동성 레짐**: 전체 시장이 **고변동성 국면인지 저변동성 국면인지** 표시하는 변수이다. 예를 들어 **VIX 지수** (S&P500 변동성 지수)가 20 이상이면 high-vol regime (1), 이하면 low-vol regime (0)으로 구분할 수 있다. 혹은 S&P500 지수의 1개월 역사적 변동성이 평소보다 높은 구간을 1로 표시할 수도 있다. 밈 주식의 contrarian 효과가 시장이 불안정할 때 더욱 두드러지거나 반대로 완화될 가능성이 있으므로, 모델이 이를 조건부로 학습하도록 돕는다.
- **유동성/거래량 레짐**: 해당 종목의 **거래량이 이례적으로 높거나 낮은 경우** 표시한다. 예를 들어 거래량이 10일 평균 대비 2배 이상이면 `is_high_volume=1`. Reddit발 관심 폭증이 실제 거래량 급증으로 이어지는 날이 많으며, 이때 가격 충격이 클 수 있다. 따라서 거래량 레짐 변수를 통해 모델이 **거래량 급증 여부**를 인지하게 할 수 있다.
- **기간/이벤트 레짐**: 데이터 기간 중 알려진 중요한 이벤트(예: 재무제표 발표일, 공매도 제한 조치 시기 등)를 반영하는 것도 가능하다. 예를 들어 2021년 1월 말 GameStop 사태 기간을 특정하여 그 주간을 별도 레짐으로 태깅하거나, 혹은 **팬데믹 기간 vs 이후 기간** 등을 나뉘볼 수 있다. 이러한 변수는 선택 사항이지만, 특정 기간에만 나타나는 구조적 패턴(예: 2021년 초의 극단적 군집행동)이 전체 학습을 교란하지 않도록 해준다.

레짐 변수를 피처로 넣으면 모델은 자동으로 그 영향을 학습할 수 있지만, 경우에 따라 **레짐별로 별도 모델을 학습**하는 전략도 고려할 수 있다. 예를 들어 스파이크 이벤트 날만 모아서 성능을 평가해보거나, 반대로 평상시 데이터로만 모델을 학습해 본 뒤 스파이크 날에 적용해보는 플라시보 실험도 가능하다 (이에 대해서는 후술한다).

결과적으로, 레짐 특성은 **"현재 시장/심리 상태가 어떤 국면인가"**를 알려주는 신호로서, Contrarian 효과의 맥락(Context)을 모델이 이해하도록 돕는 역할을 한다. 이를 통해 보다 **상황 적응적인 예측**이 가능해질 것으로 기대된다.

3. 학습 설계 (Train/Validation/Test 세트 구성 및 검증)

머신러닝 모델의 일반화 성능을 높이기 위해서는 **데이터 분할과 학습 방법**을 신중히 설계해야 한다. 특히 시계열 금융 데이터는 샘플들 간 **시간적 의존성**이 크므로, 무작위 교차검증보다는 **시계열에 특화된 검증 전략**이 요구된다. 여기에선 **학습/검증/테스트 세트 구성 방법**, 그리고 **Purged K-Fold, Walk-forward validation** 등의 기법을 다룬다 ¹⁶. 또한 **과적합 방지**와 **검증 안정성**을 위한 팁을 포함한다.

3.1 데이터 세트 분할 (Training/Validation/Testing Split)

먼저 전체 데이터를 **시간 순서대로** 세 구간으로 나눈다: - **훈련 세트 (train)** - 모델 학습에 사용. (예: 2021년~2022년 데이터) - **검증 세트 (validation)** - 하이퍼파라미터 튜닝 및 모델 선택에 사용. (예: 2023년 상반기 데이터) - **테스트 세트 (test)** - 최종 모델의 성능 평가에 사용. (예: 2023년 하반기 데이터)

시계열 데이터의 경우 검증/테스트 세트는 반드시 **훈련보다 미래 시점**이어야 하며, 훈련에 사용된 데이터의 시간이 겹치지 않도록 구분해야 한다 ¹⁷ ¹⁸. 예컨대 2023년 데이터를 테스트로 사용할 경우, 2023년의 데이터는 모델 학습에 절대 포함되지 않도록 2021-2022년까지만 훈련에 사용한다. 이렇게 하는 이유는 미래 예측 성능을 제대로 측정하고 **미래 정보 누설 (leakage)**을 방지하기 위함이다.

실제 구현에서는 데이터프레임을 날짜 기준 정렬한 뒤, 인덱스로 분할할 수 있다:

```
# data는 날짜순 정렬된 데이터프레임
train = data[data['date'] < '2023-01-01']
val    = data[(data['date'] >= '2023-01-01') & (data['date'] < '2023-07-01')]
test   = data[data['date'] >= '2023-07-01']
```

이렇게 분할한 후, **feature와 label 컬럼을 분리**하여 모델 입력 X와 출력 y를 준비한다:

```
X_train, y_train = train[feature_cols], train['next_return']
X_val, y_val     = val[feature_cols], val['next_return']
X_test, y_test   = test[feature_cols], test['next_return']
```

검증 세트는 필수가 아니지만, **모델 및 하이퍼파라미터 튜닝**을 위해 권장된다. 데이터가 충분히 많지 않다면 (예: 2년치 이하) **교차검증 기법**을 통해 검증을 대체하고, 마지막 일부만 테스트로 둘 수도 있다.

3.2 Purged K-Fold 교차검증

K-Fold Cross-Validation은 데이터를 K개 폴드로 나누어 그 중 하나를 검증, 나머지를 학습에 사용하는 과정을 K번 반복하는 기법이다. 그러나 **금융 시계열 데이터**에는 일반적인 K-Fold를 그대로 적용하면 **치명적인 오류**가 발생할 수 있다¹⁹
²⁰ : - 샘플들이 **독립**이 아니고 시계열 상 **연관**되어 있어, 임의 분할시 **미래 정보가 학습에 섞이는** 문제가 발생 (look-ahead bias). - 같은 테스트 세트를 여러 번 사용하면 튜닝과정에서 **정보 누설(selection bias)**이 생길 수 있음.

이를 해결하기 위한 방법 중 하나가 **Purged K-Fold CV**이다²¹. Purged K-Fold에서는 **검증 구간과 겹치는 훈련 샘플들을 제거(purge)**하여 미래 정보 유출을 막는다¹⁶. 즉, **테스트 기간의 레이블과 시간이 겹치는 훈련 샘플들을 모두 제거**한 후 학습을 진행한다²². 예를 들어, 5-Fold CV를 시계열 순으로 구현한다고 하면: - Fold 1: 검증세트 = 전체 기간의 앞 20%, 훈련세트 = 뒤 80%. 단, 검증 구간과 인접한 훈련샘플은 제외. - Fold 2: 검증세트 = 다음 20%, 훈련세트 = 나머지 (앞 20% + 뒤 60%) - 이때도 겹침 제거. - ... 반복 ...

여기서 겹침 제거(purging)의 범위는 **라벨에 영향주는 기간**에 따라 결정된다. 만약 우리가 t일의 **다음날 수익률**을 예측하는 모델이면, t일의 피처를 사용해 t+1일 수익률을 맞추는 구조이므로 **검증 구간의 직전날 데이터**까지는 훈련셋에서 제거해야 한다. 일반적으로 **embargo period(금지 기간)**를 설정하여 테스트 구간 전후의 데이터 일부를 훈련에서 빼는 방식을 취한다²³. 예컨대 테스트 구간이 7월 1일~7월 31일이라면, 훈련에서는 6월 하순 며칠과 7월 전체, 그리고 (필요시) 8월 초 며칠을 제거하는 식이다. 이는 **정보의 잔존 효과**(예: 라벨 누출)를 최소화하기 위함이다.

Purged K-Fold를 직접 구현하려면, 각 폴드에 대해: 1. 테스트(검증) 기간의 **시작/끝 인덱스**를 식별. 2. 훈련 데이터에서 해당 기간 + 전후 embargo 기간을 제외. 3. 남은 데이터로 모델 훈련, 테스트 기간으로 검증.

이를 수식으로 표현하면, 테스트 기간 $T_j = [t_{start}, t_{end}]$ 에 대해 훈련 집합에서 모든 $t \in [t_{start} - \delta, t_{end} + \delta]$ (여기서 δ 는 embargo 일수, 예: 5일)을 제거한다²². 이러한 과정을 K번 반복하여 평균 성능을 산출한다. **Purged K-Fold의 장점**은 미래 정보 이용을 차단하여 **낙관적 편향**을 줄이고 보다 **현실적인 검증성능**을 얻을 수 있다는 점이다²⁴.

참고: Purged CV 구현을 돕는 패키지도 존재한다 (예: *mlfinlab* 라이브러리의 PurgedKFold 클래스). 이를 활용하면 간편하게 교차검증 폴드를 생성할 수 있다. 하지만 의사코드로 직접 구현해 보는 것도 이해에 도움이 된다.

3.3 Walk-Forward (롤링) 검증

Walk-Forward Validation은 실제 투자 시계열의 진행을 모사한 검증 기법으로, **시간축을 따라 모델을 재학습**하며 여러 번의 훈련-검증을 수행한다²⁵²⁶. 예를 들어 다음과 같은 순차적 절차를 갖는다: 1. **첫 번째 훈련/검증:** 첫 N개월(또는 N일)의 데이터를 훈련세트로 사용하여 모델 학습, 그 다음 M개월을 검증. 2. **두 번째 훈련/검증:** 훈련세트를 **앞으로 이동** - 예: 1단계에서 사용한 검증기간을 훈련에 추가하거나 (확장형) 또는 동일한 크기로 슬라이딩 - 하여 모델 재학습, 그 다음 구간을 검증. 3. 이 과정을 데이터 끝까지 진행하여, 여러 검증 구간의 성능을 확인.

예시로 2021-01 ~ 2023-12 데이터를 6개월 단위 **expanding window** Walk-forward로 검증한다고 하면: - 2021-01 ~ 2021-12: 훈련, 2022-01 ~ 2022-03: 검증 - 2021-01 ~ 2022-03: 훈련 (이전 훈련+검증 합침, expanding), 2022-04 ~ 2022-06: 검증 - 2021-01 ~ 2022-06: 훈련, 2022-07 ~ 2022-09: 검증 - ... 마지막으로 2021-01 ~ 2023-09: 훈련, 2023-10 ~ 2023-12: 검증

각 검증 구간의 예측 성능을 측정하여 평균을 내거나 분포를 살펴보면, **시간에 따른 모델 안정성과 일관성**을 평가할 수 있다²⁷. Walk-forward의 장점은 모델이 **초기엔 적은 데이터로 학습**하고 차츰 **데이터가 누적**되며 학습함으로써, **데이터 증가**

에 따른 성능 변화를 관찰할 수 있다는 점이다. 또한 **실제 운영과 유사하게**, 새로운 데이터가 생기면 모델을 업데이트하는 방식이라 **실용적 성능**을 가늠할 수 있다.

구현은 루프를 사용하여 window를 슬라이딩하면 된다:

```
from datetime import timedelta
train_start = start_date
while train_end < end_date:
    # 설정한 윈도우 크기만큼 훈련/검증 구간 설정
    X_train = data[(data.date >= train_start) & (data.date <= train_end)]
    [features]
    y_train = ...
    X_val = data[(data.date > train_end) & (data.date <= val_end)][features]
    y_val = ...
    model = train_model(X_train, y_train)
    perf = evaluate(model, X_val, y_val)
    # 결과 저장 및 윈도우 이동
    results.append(perf)
    train_end = train_end + timedelta(months=3) # 예: 3개월씩 이동
    val_end = val_end + timedelta(months=3)
```

위 의사코드에서 보듯이, train_end와 val_end를 조정하면서 순차적으로 나아간다. 이때도 중요한 것은 **겹치지 않는 구간**으로 해야 하며, 앞선 검증기간의 데이터는 다음 단계에 훈련으로 포함(확장형)하거나 제외(슬라이딩 창)할지 결정해야 한다. 일반적으로 데이터가 많지 않다면 **확장(expanding)**이 안정적이며, 데이터가 풍부하고 변화가 심한 경우 **슬라이딩(rolling window)**으로 최근 데이터만 사용하기도 한다.

3.4 모델 튜닝과 과적합 방지

데이터 분할과 검증 전략을 마련했다면, 이를 활용하여 **하이퍼파라미터 튜닝**을 진행한다. LightGBM이나 XGBoost 같은 모델의 경우, **교차검증**이나 **Walk-forward**의 결과에서 **평균 성능이 최적인** 파라미터를 선택한다. 튜닝해야 할 주요 파라미터로는: - 트리 모델: num_leaves (리프 수), max_depth (트리 깊이), learning_rate, n_estimators (트리 개수), subsample / colsample_bytree (샘플링 비율), reg_alpha, reg_lambda (정규화). - 딥러닝 모델: 뉴런 수, 레이어 수, 학습률, 배치 크기, epochs 등이 있다.

튜닝 시 유의할 점은 **과적합을 피하기 위해 보수적으로 선택**하는 것이다²⁸. 예를 들어 트리의 max_depth를 너무 깊게 하지 않고 (3~5 정도), learning_rate도 너무 작게 잡지 않으며 (0.01~0.1 범위), 정규화 파라미터는 충분히 크게 준다. 또한 **Early Stopping**을 활용하여 검증손실이 일정 epochs 동안 개선되지 않으면 학습을 중단한다. 딥러닝의 경우 **드롭아웃(dropout)**과 **배치 정규화(batch normalization)** 등의 기법도 과적합 억제에 도움이 된다²⁹³⁰.

Purged K-Fold나 Walk-forward로 얻은 **검증 성능 (예: IC, RMSE 등)**을 비교하여, 어느 파라미터 세팅에서 일관되게 좋은지 판단한다. 금융데이터는 노이즈가 많아 **한 번의 실험 결과에 일희일비하지 말고**, 여러 기간에 걸쳐 안정적으로 성능이 나는 설정을 선택하는 것이 중요하다. 또한 **모델 선택**도 이 단계에서 이뤄진다. 만약 여러 종류의 모델(예: LightGBM vs XGBoost vs Neural Net)을 후보로 두었다면, 동일한 검증 방법으로 성능을 재면서 가장 우수한 것을 고른다.

튜닝을 마친 최종 모델은 **테스트 세트**에 한 번만 적용하여 성능을 평가한다. 이때는 절대로 **테스트 성능을 기준으로 모델이나 파라미터를 다시 조정하면 안 된다** (정보 누설). 테스트 결과는 어디까지나 최종 확인용으로 사용하고, 보고되는 모든 지표 (예: Sharpe, Hit-rate 등)는 이 **오직 한 번의 테스트 평가**에서 산출하도록 한다. 그래야만 연구 결과의 **신뢰성**을 담보할 수 있다.

마지막으로, 데이터 분할과 검증 과정을 도식화하면 다음과 같다:

```
|<----- 전체 기간 데이터 ----->|
| Train .....[ Val ....][ Test ..]
|<-- Purged K-Fold CV -->| (최종평가)
|<-- Walk-forward splits (훈련->검증 반복) -->|
```

위 그림에서 보듯 다양한 검증을 거쳐 모델을 개선한 후, 최종적으로 테스트로 검증하면 된다. 이러한 체계적인 학습 설계는 **일반화 성능**을 높이고, 운에 좌우되지 않는 **견고한 성과**를 얻는 데 필수적이다.

4. 모델 구조: Baseline 모델 vs 향상된 모델

기본 특성과 검증 방식을 정했다면, 이제 **어떤 모델을 쓸 것인가**의 문제이다. 본 연구의 기본 논문에서는 **릿지 회귀 (선형회귀)**, **LightGBM**, **XGBoost**의 세 가지를 비교하였고, 소셜미디어 contrarian 효과를 고려하지 않은 경우 이들의 예측이 실제와 **음의 상관관계**를 보였으나, 결과에 -1을 곱하는 contrarian 전략 적용 후 상관계수가 **양(+)**전환되며 성능이 향상되었다^{31 32}. 여기서는 이러한 **Baseline 모델 구조**와 이를 넘어서는 **향상된 모델 구조**들을 살펴본다. 특히 **Temporal Convolutional Network (TCN)**, **Temporal Fusion Transformer (TFT)**, **메타 앙상블(meta-ensemble)** 기법 등 **시계열 딥러닝과 앙상블** 기법을 도입하여 성능 향상을 모색한다.

4.1 기본 모델 (Baseline: 선형 및 트리 계열)

- **선형 회귀 (Ridge Regression)**: 가장 해석이 쉽고 빠른 모델로, 주어진 모든 특성의 **선형 결합**으로 목표를 예측한다. 릿지 회귀는 L2 정규화로 과적합을 줄인 버전으로, 각 특성의 **계수 값**을 통해 방향성과 중요도를 파악하기 쉽다. Baseline으로서 릿지를 사용하면, Reddit 관련 특성들의 계수가 음수로 학습되는지 (즉 언급량 ↑ → 수익률 ↓ 관계 학습) 등을 확인할 수 있다. 다만 선형모델은 복잡한 비선형 패턴을 캡처하지 못하므로 성능은 제한적일 수 있다.
- **LightGBM (Gradient Boosting 결정나무)**: 트리 기반 모델로, 비교적 적은 튜닝으로도 강력한 성능을 내는 **앙상블 결정나무** 모델이다. 본 케이스에서는 LightGBM이 contrarian 신호 적용 후 높은 IC 향상(+0.108포인트)과 Hit-rate 개선을 보였으며³¹, 이는 비선형 조합을 일부 포착했기 때문으로 해석된다. LightGBM의 장점은 **피처 중요도**를 바로 제공하고, **결측치나 이상치에 강건** 하며, **학습속도**가 빠르다는 것이다. Baseline 단계에서는 LightGBM의 하이퍼파라미터를 비교적 보수적으로 설정한다. 예를 들어:

- `num_leaves=31` (작은 리프 개수로 과적합 방지)
- `max_depth=5` (트리 깊이 제한)
- `learning_rate=0.05` (적당한 학습률)
- `n_estimators=200` (트리 200개; early_stopping으로 자동 조절)
- `subsample=0.8, colsample_bytree=0.8` (샘플, 피처 부분 사용)

등의 값으로 시작해볼 수 있다. 훈련은 Python에서 다음과 같이 진행 가능하다:

```

import lightgbm as lgb
train_data = lgb.Dataset(X_train, label=y_train)
valid_data = lgb.Dataset(X_val, label=y_val)
params = {
    'objective': 'regression',
    'metric': 'rmse', # 기본 평가지표 (추후 custom eval로 IC 등 활용 가능)
    'num_leaves': 31,
    'max_depth': 5,
    'learning_rate': 0.05,
    'feature_fraction': 0.8,
    'bagging_fraction': 0.8,
    'seed': 42
}
model = lgb.train(params, train_data, valid_sets=[valid_data],
                  num_boost_round=500, early_stopping_rounds=20)

```

이처럼 **조기중단(Early Stopping)**을 적용하여 최적 트리 수를 자동 결정한다. Baseline 모델로서 LightGBM은 구현과 해석이 비교적 쉽고, 성능도 준수하여 출발점으로 적합하다.

- **XGBoost:** LightGBM과 유사한 그래디언트 부스팅 트리 모델로, 약간 다른 분할 방식과 최적화 알고리즘을 사용한다. 두 모델의 성능은 데이터셋에 따라 엇비슷하나, XGBoost가 **과적합 제어**에 조금 더 보수적일 수 있다. 논문 결과에서는 contrarian 적용 후 XGBoost의 Hit-rate가 0.384 → 0.448로 크게 향상되어, LightGBM보다 **방향예측(Hit) 개선폭이 컸다** ³¹. 이는 XGBoost 모델이 contrarian 효과를 반영했을 때 잘 맞추는 날이 늘었음을 시사한다. Baseline 튜닝값은 LightGBM과 유사하게 설정할 수 있다 (`max_depth=5`, `eta=0.05`, `subsample=0.8` 등). Python 구현도 LightGBM과 비슷하게 `xgboost.DMatrix`를 사용하거나 `sklearn` API(`XGBRegressor`)를 사용할 수 있다.

Baseline 모델들의 훈련 후에는 **검증세트 결과**를 확인한다. 만약 baseline 모델이 **음(-)의 IC**를 보인다면 (실제로 LightGBM의 초기 예측과 실제 수익률 간 상관관계가 약 -0.05였음 ³²), 이는 모델이 방향을 거꾸로 잡고 있다는 뜻이므로 contrarian 전략을 적용하면 개선될 가능성이 크다. 하지만 **이러한 뒤집기만으로 만족하지 말고**, 다음 섹션처럼 모델 자체가 contrarian 관계를 학습하도록 고도화할 수 있다.

4.2 향상된 모델 구조 (고급: 딥러닝, 앙상블 등)

- **Temporal Convolutional Network (TCN):** TCN은 시계열 데이터를 처리하기 위한 1D 합성곱 신경망으로, **길어진 과거 시퀀스도 학습할 수 있도록 커널의 지수적 확장(dilated CNN)**과 **인과적 구조(causal conv)**를 갖는다 ³³ ³⁴. RNN/LSTM 대비 **병렬처리 효율**이 높고 **장기 의존성 학습**에 유리하여, 금융 시계열에도 점차 활용되고 있다. TCN 구조는 예컨대 다음과 같다:
 - 입력: 최근 20일치의 모든 특성 시퀀스 ($X_{t-19}, \dots, X_t$)를 한 슬라이딩 윈도우로 취함.
 - 첫 번째 1D Convolution 레이어: 필터 크기 3, dilation=1 (직전 시점들 참조)으로 수십 개의 채널 추출, 활성화 함수 ReLU.
 - 두 번째 Conv 레이어: 필터 크기 3, dilation=2 (2시점 건너뛰며 참조), ReLU.
 - 세 번째 Conv 레이어: 필터 크기 3, dilation=4 (4시점 건너뛰며 참조), ReLU.
 - ... (필요시 더 깊게) ... 각 합성곱 블록 후에 BatchNorm과 Dropout(예: 0.2) 적용으로 안정화 ²⁹ ³⁰.
 - 마지막으로 Flatten 후, Dense(fully-connected) 출력층으로 다음날 수익률 예측.

이런 TCN 모델은 **과거 2^k 기간까지** 정보에 접근할 수 있어 (dilation이 1,2,4,... 진행 시), 예컨대 3계층이면 최대 4 이전의 receptive field -> 7 (중첩효과로 더 커짐) 등 장기 메모리를 확보한다. LSTM과 비교하면 **기울기 소실이 덜하고** 속도가 빠르며 ³⁵ ³⁶, 시퀀스가 긴 경우에도 잘 작동한다.

우리 문제에 TCN을 적용하면, **특성 공학을 통해 얻은 여러 시계열** (가격, 언급량, 감정 등)의 **패턴 조합**을 모델 스스로 학습하게 할 수 있다. 예를 들어 "지난 열흘간 언급량이 급증하고 가격은 하락 추세" 같은 복합 패턴을 TCN이 필터를 통해 인식할 수 있다. 이를 통해 baseline에서 단순 lag 피쳐로 제공한 정보 이상의 **비선형 패턴** 학습이 가능해져 예측력을 높일 수 있다.

Python 구현은 Keras 등을 통해 가능하다. 앞서 2장에서 생성한 feature sequence를 사용하여 (batch_size, time_window, feature_dim) 형태의 입력을 준비한다. 모델 구성 예시는 다음과 같다:

```
from keras.models import Sequential
from keras.layers import Conv1D, BatchNormalization, ReLU, Dropout, Flatten, Dense
model = Sequential()
model.add(Conv1D(filters=64, kernel_size=3, dilation_rate=1, padding='causal',
activation='relu', input_shape=(20, num_features)))
model.add(BatchNormalization()); model.add(Conv1D(64, 3, dilation_rate=2,
padding='causal', activation='relu'))
model.add(BatchNormalization()); model.add(Conv1D(64, 3, dilation_rate=4,
padding='causal', activation='relu'))
model.add(Flatten()); model.add(Dropout(0.2))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mse')
```

(실제론 Residual 연결 등 TCN의 권장 구조가 더 있지만, 단순화하여 예시로 제시). 이 모델을 이전 섹션의 train/val 세트 시퀀스에 대해 학습시키고, 조기중단 및 학습률 스케줄링을 적용한다 ³⁷ ³⁸. 기대 효과는 baseline보다 낮은 RMSE와 **높은 IC**를 얻는 것이다. 특히 TCN은 **비선형 관계**를 포착하므로, contrarian 효과(음의 관계) 역시 비선형 상황에서 더 정확히 잡아줄 수 있다.

- **Temporal Fusion Transformer (TFT)**: TFT는 **애플리케이션 맥락에 맞게 설계된 시계열용 어텐션 기반 모델**로, 복잡한 다변량 시계열에서 **중요한 시점과 특성**에 가중치를 할당하여 예측하는 아키텍처다 ³⁹. TFT의 구성은 크게 세 부분으로 나뉜다:
- **가변적 입력 처리**: 과거 구간 입력(historical inputs)과 미래에 미리 아는 공휴일 등의 정보(future known inputs)를 각각 LSTM 등을 통해 인코딩.
- **인터프리터블 어텐션 블록**: 여러 시점의 정보를 self-attention 메커니즘으로 종합하여, 예측에 중요한 시점/변수에 높은 어텐션 가중치를 부여 ³⁹.
- **출력 계층**: multi-head attention 출력과 skip connection 등을 거쳐 최종 예측 산출.

TFT는 **특성 중요도**와 **시점 중요도**를 모두 어텐션 가중치로 제공하므로, 예측 결과에 어떤 요인이 크게 기여했는지 해석하기 용이하다. 예컨대 TFT를 훈련하고 나면 "언급량 특성이 최근 3일 중 1일에 크게 작용하여 하락 예측에 기여했다"는 식의 해석이 가능하다. 이러한 **모델 내장 해석력**은 연구자에게 contrarian 메커니즘을 검증하는 추가 단서를 줄 수 있다.

기술적 구현 측면에서, PyTorch의 pytorch-forecasting 패키지에 TFT 구현이 있어 비교적 쉽게 활용할 수 있다 ⁴⁰. 해당 라이브러리를 쓰면 TemporalFusionTransformer 클래스를 사용하여 max_encoder_length,

max_prediction_length 등을 설정하고, 지정한 feature set을 학습시킬 수 있다. 우리 문제에서는 **하루 앞** 예측이므로 prediction_length=1, encoder_length=예컨대 20로 설정하면 된다. 하이퍼파라미터로는 **hidden layer dimension** (예: 32 또는 64), **attention head 수** (예: 4), **dropout** (예:0.1) 등을 조절해볼 수 있다.

TFT의 기대 성능은 TCN과 유사하거나 그 이상인데, 특히 여러 변수간 **상호작용**이 복잡할 때 효과적이다. 또한 **훈련이 잘 되면** 아주 낮은 오차율로 예측이 가능했다는 보고도 있다 (인도네시아 주식 데이터에서 SMAPE 0.0022 등 ⁴¹). 다만 주식 데이터는 잡음이 많아 과적합 위험이 있으므로, 충분한 regularization과 검증을 거쳐야 한다.

- **메타-앙상블 (Meta-Ensemble):** 여러 모델을 결합하여 최종 예측을 산출하는 방법이다. 앙상블의 논리는 서로 다른 모델들이 각기 다른 오류 패턴을 가지므로 결합하면 **평균 오차는 줄이고 일관된 신호는 강화**할 수 있다는 것이다 ⁴² ⁴³. 본 연구 맥락에서 고려할 앙상블 기법:
 - 단순 앙상블: Baseline 모델 (LightGBM, XGBoost)과 딥러닝 모델 (TCN, TFT)의 예측값을 **평균**하거나 **가중 평균**하여 최종 예측으로 사용. 가중치는 검증 세트 성능에 비례하여 줄 수 있음 (예: 검증 IC가 높은 모델에 더 가중).
 - 스택킹(Stacking): 1단계로 여러 모델의 예측치를 모아 (예: LightGBM_pred, TCN_pred, TFT_pred) 2단계 메타 모델의 입력으로 사용하고, 실제 값을 메타모델이 학습하도록 함. 메타모델은 간단히 **선형 회귀**나 **로지스틱 회귀**(방향 예측이면)로 할 수 있다. 이를 통해 각 모델의 신뢰도를 자동으로 학습하게 된다.
 - 베이지안 앙상블: 모델 예측을 확률해석하여 **Bayesian Model Averaging** 등을 적용하는 방법도 있으나, 구현이 복잡하므로 언급만 한다.

앙상블은 특히 **예측 변동성 감소**와 **성능 안정화**에 효과적이다. 예를 들어 LightGBM은 특정 패턴에 강하지만 비선형 긴 시계열 패턴을 놓칠 수 있고, 반대로 TCN은 시계열 패턴에 강하지만 극단치 처리에서 트리보다 약할 수 있다. 둘을 평균하면 극단치 상황에서도 완화된 예측을 얻을 수 있다. 실제로 **Bagging(배깅)** 등 앙상블은 과적합을 줄이고 일반화 성능을 높이는 것으로 알려져 있다 ⁴⁴ ⁴⁵.

주의할 점은, 앙상블을 쓴다고 해서 **정보 누설**이 생겨서는 안 된다는 것이다. 따라서 메타모델 훈련 시에도 마찬가지로 **훈련/검증 분리**를 지켜야 한다. 예를 들어 1단계 모델들을 훈련세트에서 학습시키고, **그들의 검증세트 예측 결과**를 모아 메타모델을 훈련한다 (또는 cross-val blending 기법 사용). 최종 테스트에서는 각각의 모델이 낸 예측을 같은 방식으로 조합해준다.

- **기타 고려 모델:** 만약 데이터가 충분하고 특징이 많다면 **랜덤포레스트**나 **다층 퍼셉트론(MLP)** 등의 간단한 모델도 시도해볼 수 있다. 하지만 랜덤포레스트는 시계열 패턴 포착이 어려워 성능이 떨어질 수 있고, MLP는 TCN/TFT 대비 시계열 구조 활용이 미흡하다. 차라리 **LSTM/RNN** 계열이 과거에는 많이 쓰였으나, TCN/TFT가 전반적으로 성능과 훈련 안정성에서 우수하므로 본 매뉴얼에서는 권장하지 않는다.

마지막으로 **Baseline vs 앙상 모델의 비교**를 정리하면: - Baseline: (Linear/Ridge) + (GBM들). 빠르고 해석 쉬우나, contrarian 비선형 효과 직접 학습 한계. - 앙상: (TCN/TFT 딥러닝) + (Ensemble). 복잡한 패턴 학습 가능, 잠재적 성능 향상. 단, 데이터가 부족하면 오히려 **과적합 위험** 있으므로 충분한 검증 필수.

Tip: 처음에는 Baseline으로 간단한 모델부터 시도해서 **Contrarian** 신호가 얼마나 잡히는지 확인하고, 점차 고도화하는 것을 권장한다. 예를 들어 LightGBM으로 어느 정도 IC가 개선되었다면, 그 다음 TCN을 도입해보는 식으로 진행한다. 각 단계마다 **검증 성능과 기존 연구 결과의 정합성** (예: 음->양 상관 전환 등)을 검토하며 나아가면, 일관성과 성능을 모두 잡을 수 있다.

5. 손실 함수 및 평가 지표

모델을 학습시킬 때 무엇을 최적화하고, 성능을 무엇으로 평가할 것인가는 연구의 성공을 가르는 중요한 요소이다. **손실 함수(loss)**는 모델이 학습 중 최소화하려는 목표이며, **평가 지표(metric)**는 학습 후 모델을 객관적으로 평가하는 기준이다. 금

응시계열 예측에서는 일반적인 회귀 오차 외에 **랭킹 관점의 지표**나 **투자 성과 지표**를 활용하는 것이 유용하다. 여기서는 **RankIC 손실, Hit-rate(방향 정확도), Sharpe Ratio, Turnover, Max Drawdown** 등을 다룬다.

- **평균 제곱오차 (MSE) 및 MAE:** 우선 기본 손실 함수로 MSE 또는 MAE(Mean Absolute Error)를 사용할 수 있다. MSE는 $\frac{1}{N} \sum (y_{\text{pred}} - y_{\text{true}})^2$ 로 계산되며, 연속형 수익률 예측에서 많이 쓰인다. MAE는 절대 오차로 $\frac{1}{N} \sum |y_{\text{pred}} - y_{\text{true}}|$ 이다. Baseline 모델들은 기본적으로 MSE를 최소화하도록 학습시켰다. 그러나 MSE를 최소화한다고 반드시 투자 성과가 최대화되는 것은 아니다 - 예컨대 MSE는 큰 오차에 패널티를 많이 주므로 극단값 예측을 조심하게 하지만, 방향이 반대인 작은 오차 vs 같은 방향의 큰 오차에 대한 구분이 없다.
- **RankIC (Rank Information Coefficient) 기반 손실:** RankIC란 **예측값과 실제 결과 간의 스피어만 순위상관계수**를 뜻한다 ⁴⁶. 구체적으로, 일련의 예측과 실제 수익률을 각각 순위로 변환한 뒤 피어슨 상관계수를 계산한 값이다. RankIC는 수익률의 **정확한 크기보다 순서**에 초점을 맞추므로, 투자 전략 관점에서 **종목 혹은 시점 간 랭킹**을 제대로 맞추는지를 평가한다. 이를 손실에 적용하려면, **RankIC를 높이는 방향으로** 학습을 유도해야 한다. 한 가지 방법은 **손실 함수에 RankIC 관련 항(term)을 추가** 하는 것이다 ⁴⁷ ⁴⁸. 예를 들어, 기본 MSE 손실을 L_{MSE} , RankIC를 r 이라고 하면, 최종 손실을

$$L_{\text{total}} = L_{MSE} - \lambda \cdot r$$

처럼 정의할 수 있다 (RankIC는 높을수록 좋으므로 음의 기여로 넣음). 여기서 λ 는 RankIC의 중요도를 조절하는 하이퍼파라미터이다. 실제 구현 시 RankIC는 미분가능하지 않은 지표라서 바로 최적화하기 어려운데, 이를 근사하기 위해 **pairwise ranking loss**를 도입하기도 한다. 예를 들어 두 샘플 i, j 에 대해 $y_i > y_j$ 이면 $y_{\text{pred},i}$ 도 더 크게 나오도록, **랭킹 SVM**이나 **람다랭크(lambdaRank)** 스타일의 손실을 부분적으로 적용하는 것이다. LightGBM의 경우 `objective='lambdarank'` 를 설정하면 (원래 정보검색용이지만) 유사하게 순위손실을 최적화할 수 있다. 다만 이때 group을 적절히 설정해야 하며, 일반 시계열에는 바로 적용이 어려울 수 있다 (크로스섹션 랭킹에 적합).

현실적인 접근은 **모델 훈련 후** RankIC를 높이는 방향으로 **하이퍼파라미터 튜닝**을 하는 것이다. 예를 들어 여러 파라미터 조합 중 검증세트 RankIC가 가장 높은 모델을 선택하면, 결과적으로 RankIC를 간접적으로 최적화한 셈이 된다. 또한 Early stopping 시 평가 지표로 RankIC를 사용할 수도 있다. LightGBM은 custom eval metric을 지원하므로, 아래처럼 구현할 수 있다:

```
import numpy as np
def rankic_eval(preds, train_data):
    y_true = train_data.get_label()
    # Spearman rank correlation
    rank_pred = np.argsort(np.argsort(preds))
    rank_true = np.argsort(np.argsort(y_true))
    corr = np.corrcoef(rank_pred, rank_true)[0,1]
    return 'rankIC', corr, True # True-> higher is better
model = lgb.train(params, train_data, valid_sets=[valid_data],
                  feval=rankic_eval, early_stopping_rounds=10)
```

위 함수는 검증셋의 RankIC를 계산해 학습 중 모니터링한다. RankIC를 직접 loss로 쓰진 않지만, 참고지표로 두어 모델이 음의 상관으로 가는 것을 방지할 수 있다. 실제 연구 ²⁷ ⁴⁹ 에서도 RankIC 정규화를 도입한 모델이 value accuracy(RMSE)는 조금 희생해도 순위 관련 지표(IC/RankIC)가 크게 향상되고, 이에 따라 투자 성과가 좋아졌음을 보여준다.

• **Hit-Rate (방향 정확도):** 이는 예측값의 부호(sign)가 실제 부호와 맞았는지의 비율이다. 즉, **내일 주가상승/하락을 맞췄는가?**를 나타낸다. 수식으로 Hit-rate = $\frac{1}{N} \sum \mathbf{1}[(y_{\text{pred}} > 0) = (y_{\text{true}} > 0)]$ 로 정의할 수 있다 (동일부호이면 1, 아니면 0인 indicator의 평균). 논문에서도 Hit-rate를 주요 성능 지표로 보고하였는데, 예를 들어 LightGBM의 Hit-rate가 contrarian 적용 후 0.406 -> 0.426 (42.6%)로 향상되었음을 확인했다 ³¹ ⁵⁰. Hit-rate는 **클래스 균형**에 영향을 받으므로, 상승/하락 비율이 50:50이 아닐 경우 해석에 주의해야 한다. 예컨대 밌주식이 전체 기간 55% 하락일수가 많다면, 한쪽으로만 예측해도 55% 정확도가 나오기 때문에, 항상 **벤치마크(예: 무조건 하락 예측)**와 비교해서 봐야 의미가 있다. 그럼에도 Hit-rate는 투자자 입장에서 **맞힌 날의 비율**이란 직관적인 의미가 있어 보고 가치가 있다. 손실함수로 Hit-rate를 직접 쓰긴 어렵지만(비연속적), 훈련 중 참고지표로 모니터링하며 높이는 방향으로 튜닝할 수 있다.

• **Sharpe Ratio (샤프 지수):** 샤프 지수는 **투자 전략의 단위위험당 초과수익률**을 뜻하는 지표로, 주로 연율화하여 사용한다. $\text{Sharpe} = \frac{E[R - R_f]}{\sigma(R)}$ 인데, 무위험 이자율 R_f 를 보통 0으로 두고 단순화해 $\frac{\text{평균 일간수익률}}{\text{표준편차}} \times \sqrt{252}$ (일별 데이터 기준)로 계산한다. 모델이 매일 예측에 따라 롱/숏 포지션을 취하는 전략을 구현했을 때, 그 **일일 수익률 시계열**로 Sharpe Ratio를 산출한다. 예를 들어 테스트 기간 일일 전략 수익률이 `r_t` 라면:

```
import numpy as np
r = np.array(strategy_returns) # 일별 전략 수익률 리스트
sharpe = (r.mean() / r.std()) * np.sqrt(252)
```

로 계산한다 (252는 연간거래일수). Sharpe이 1보다 높으면 꽤 우수한 전략으로 간주되고, 2 이상이면 매우 훌륭한 성과로 본다. Sharpe를 높이는 것은 궁극적으로 **안정적이고 우상향인 수익곡선**을 만든다는 뜻이다. Sharpe를 직접 모델 훈련에 사용하려면 강화학습 등으로 접근해야 해서 복잡하지만, 본 연구에서는 **최종 평가 지표**로 사용한다. 여러 모델을 비교할 때 Sharpe가 가장 높은 모델이 최종적으로 가장 바람직한 것으로 볼 수 있다 (물론 거래비용 및 현실적 제약을 반영해야 함은 다음에 논의).

• **Turnover (포트폴리오 회전율):** Turnover는 **거래 빈도와 포지션 변경 정도**를 나타내는 지표다. 일별로 보면, 전일 포지션과 당일 포지션의 차이를 측정해 **전체 자산 대비 거래된 비율**을 나타낸다. 간단히 말해 **하루에 포트폴리오의 몇 %를 갈아엎었나**이다. 예를 들어 전날 종목 A 100주 롱에서 오늘 100주 숏으로 바꿨다면 200주의 매매가 일어나므로 200%의 턴오버가 발생한 셈이다. 일반적으로:

$$\text{Turnover} = \frac{1}{T} \sum_{t=1}^T \sum_i |w_{i,t} - w_{i,t-1}|$$

(포트폴리오 비중 w 로 계산)로 정의한다. 우리처럼 각 날 **롱/숏 100% or 0%** 하는 전략이면, 포지션이 바뀔 때마다 2 (100% 청산 후 100% 반대포지션) 또는 1 (100% 청산 후 현금) 등의 변화가 생긴다. Turnover가 높을수록 **거래 비용**의 영향이 커지고, 실현 성과가 악화될 수 있다. 따라서 좋은 전략은 낮은 Turnover로 높은 Sharpe를 내는 것이다.

Turnover는 모델 성능 지표라기보다 **전략의 특성 지표**이므로, 결과 분석 단계에서 제시하면 좋다. 만약 두 모델이 유사한 Sharpe를 내지만 한쪽 Turnover가 훨씬 낮다면, 그 모델이 더 실용적이라고 판단할 수 있다. Turnover를 낮추기 위한 모

델 측면의 기법으로는 **예측에 히스테리시스 적용** (약한 신호에는 거래하지 않고 강한 신호에만 거래) 등이 있다. 이는 6장에서 설명될 **포지션 규칙**과 연관된다.

- **MDD (최대 낙폭, Max Drawdown):** MDD는 전략 누적수익곡선이 가장 크게 하락한 폭을 말한다. 예를 들어 100에서 시작해 150까지 갔다가 120으로 떨어졌다면, 일단 drawdown은 -20% (150 대비 120)이다. 그 후 180까지 올라갔다가 130으로 떨어지면 drawdown은 -27.8% (180 대비 130)이고, 이 중 최대인 27.8%가 MDD가 된다. MDD는 **최악의 손실 시나리오**를 나타내는 지표로, 투자자들은 Sharpe와 함께 중요하게 본다. 아무리 평균수익이 높아도 MDD가 -80%라면 실제 운용이 어려울 수 있기 때문이다.

MDD는 일별 전략 수익률 누적 곡선을 만들어 계산한다. Pandas로 간단히 구하면:

```
cum_returns = (1 + r).cumprod()
drawdown = cum_returns / cum_returns.cummax() - 1
mdd = drawdown.min()
```

로 구할 수 있다 (cummax는 이전까지의 최고점, 그 대비 현재 얼마나 하락했는지). MDD 역시 낮을수록 좋으며, 0에 가까울수록 좋다.

모델별로 전략을 구성하여 MDD를 비교하면 **리스크 관점 성능 비교**가 된다. 가령 A모델 전략 Sharpe 1.2, MDD -10%, B모델 Sharpe 1.2, MDD -30%라면, 평균 성과는 같지만 B모델은 위험이 높다고 판단할 수 있다.

- **기타 지표:** 정보계수(IC)는 Pearson 상관계수로 RankIC와 유사하지만 실제 수치 상관을 본다. IR(Information Ratio)은 IC의 평균을 표준편차로 나눈 값으로 기간별 안정성을 본다. Precision/Recall이나 AUC 같은 분류 지표는 히트율과 유사해서 잘 안 쓰지만, 특정 상황(예: 급등 맞추기)엔 활용 가능하다.

본 연구에서는 **평가 단계에서 IC, Hit-rate, Sharpe, MDD, Turnover** 등을 모두 계산하여 모델 간 비교할 것을 권한다. 이러한 지표들은 각기 다른 측면 (상관 정확도, 방향 정확도, 위험조정수익, 최대손실, 거래빈도)을 나타내므로, 종합적으로 살펴야 모델의 우열과 특성을 판단할 수 있다. Table 2에 이 지표들의 정의를 요약한다:

지표	정의	해석
IC (정보계수)	Pearson 상관계수	예측과 실제수익률 간
RankIC	Spearman 상관계수	예측과 실제수익 순위간
Hit-rate	예측방향 = 실제방향 비율 (%)	예측방향 = 실제방향 비율 (%)
Sharpe Ratio	$= \frac{\bar{r}}{\sigma(r)} \sqrt{252}$	일간 전략수익률의 평균/표준편차 (연율)
Max Drawdown	누적수익곡선 최고점 대비 최저점 하락폭 (%)	작을수록 좋음. -5% 이내 매우 안정, -20% 이하면 위험 높음
Turnover	$\frac{1}{T} \sum w_t - w_{t-1} $	일별 포트 변경 비율 합 평균
	0.5 = 하루평균 50% 포트 변경	작을수록 거래 적음.

이들 지표는 서로 보완적인 정보를 준다. 예컨대 **IC**가 높아도 **Hit-rate**는 낮을 수 있다 (순위는 맞지만 방향은 절반밖에 못 맞출 수도). 따라서 **목표에 맞는 지표에 우선순위를 두어야** 한다. 만약 **수익률 크기 예측** 자체보다는 **롱/숏 방향만 중요**하다면 Hit-rate를 더 중시하고, **포트폴리오 구성**이 목적이면 RankIC나 Sharpe를 중시한다.

마지막으로, 연구 결과 제시 시 **통계적 유의성 검증**도 수행하는 것이 바람직하다. 예를 들어 IC나 Hit-rate가 0인 귀무가설 하에서 해당 모델의 관측치가 유의한지 (t-검정이나 Diebold-Mariano test 등) 확인하면 설득력을 높일 수 있다. Sharpe에 대해서도 백테스트를 여러 번 반복(부트스트랩)하여 **신뢰구간**을 제시하면 좋다.

6. 예측 결과 기반 트레이딩 전략 적용

머신러닝 모델의 궁극적인 활용은 **예측에 따라 실제 매매를 수행** 하는 것이다. 이 절에서는 훈련된 모델의 예측값을 이용해 **트레이딩 전략**을 구성하고, **포지션 규칙**과 **거래 비용 반영** 방법을 다룬다. 또한 앞서 정의한 성과 지표들을 **전략 시뮬레이션**을 통해 도출하는 방법도 함께 설명한다.

6.1 전략 설계 기본 원칙: 본 연구 상황에서 모델은 **다음 날 수익률**(혹은 상승확률)을 예측하므로, 자연스럽게 **단기 모멘텀/반전 전략** 형태로 활용할 수 있다. Contrarian 효과를 포착한 모델이라면, 높은 소셜미디어 관심(과열)일수록 **음의 예측**(주가 하락)을 낼 것이고, 반대로 관심 식었을 땐 약간 상승 예측을 낼 수 있다. 이를 바탕으로, **매 거래일 장 마감 시점**에 다음과 같은 의사결정을 한다. - 모델의 **예측값** $\hat{y}_t$ (t일 증가 대비 t+1일 증가 수익률 예측)을 계산한다. - **포지션 결정:** $\hat{y}_t$ 가 충분히 크면 **매수(long)**, 충분히 작으면 **공매도(short)**, 그 외에는 **중립 혹은 현금 유지**로 규칙을 정한다. 간단히 $\hat{y}_t > 0$ 이면 100% 롱, $\hat{y}_t < 0$ 이면 100% 숏으로 할 수 있다. 만약 **분류 모델**로 접근했다면 바로 상승/하락 시그널로 쓰면 되고, **회귀 모델**인 경우 임계값을 0으로 두거나 **비용을 고려해 약간 폭을 뒤** 결정한다 (예: $\hat{y}$ 가 +0.1% 이상이면 롱, -0.1% 이하이면 숏, 그 사이면 포지션 유지). - **실제 매매:** t일 증가에 해당 포지션을 취한다고 가정한다. (혹은 t일 증가와 매우 근접한 가격에 다음날 시초에 매매해도 비슷한 효과로 볼 수 있다. 이벤트 드리븐 전략은 장중 매매도 고려하지만, 여기서는 일간 주기로 단순화한다.) - 다음 날(t+1) 장 마감 시 포지션을 청산 또는 조정한다. 모델이 매일 새로운 예측을 내므로, 매일 포지션을 재평가한다.

6.2 포지션 관리 규칙: 위의 기본 원칙을 실제로 적용할 때 세부 고려사항: - **부분 포지션 / 확률적 접근:** 예측값의 신뢰도에 따라 포지션 크기를 조정할 수 있다. 예를 들어 $\hat{y}_t = +2\%$ 처럼 매우 강한 신호일 때는 100% 롱, $\hat{y}_t = +0.5\%$ 처럼 애매하면 50%만 롱 등 **비례배분**할 수 있다. 하지만 비례배분은 예측 오차가 클 경우 위험하므로, Threshold를 두고 0%/100%처럼 명시적으로 구분하거나, **아예 분산투자** 측면에서 여러 종목에 신호 강도 비례로 자금을 분배하는 식으로 활용하는 것이 일반적이다. - **현금 또는 중립 포지션:** 반드시 롱 또는 숏을 들어야 하는 것은 아니다. 특정 날 예측이 거의 0이거나 자신 없으면 포지션을 닫고 현금을 보유(혹은 시장중립 유지)할 수 있다. 이를 통해 **불확실한 날의 거래비용**과 **실수 위험**을 줄인다. 예측 분포의 중앙부 구간에 **중립 밴드(no-trade region)**를 설정하는 기법은 Turnover를 낮추고 Sharpe를 높이는 데 도움이 될 수 있다. - **포트폴리오 관점:** GME, AMC, BB 등 여러 종목에 대한 예측이 있다면, 각 종목별로 위 규칙을 적용해 **동시에 운용**할 수 있다. 이 경우 **분산 투자 효과**로 전체 Sharpe가 올라갈 수 있다. 다만 이 연구의 주제는 각 밈주식의 contrarian이므로, 종목별로 독립적으로 보는 것이 일반적이다. 그럼에도 성과 평가 때는 여러 종목 전략을 **평균 합산 포트폴리오**로 묶어 Sharpe를 계산하면 더 안정적인 통계가 나온다. 실제 구현은, 하루에 n개 종목에 대해 모델 시그널이 있으면 equal-weight (혹은 신호 세기 비중)으로 롱/숏하고, 그들의 수익률 평균을 그 날 포트 수익률로 삼는다. - **포지션 연속성:** 하루 단위로 포지션을 항상 풀고 새로 잡는 것이 아니라, **기존 포지션을 유지할지 변경할지**를 결정하는 방식으로 짜면 Turnover가 감소한다. 예를 들어 어제 롱이었는데 오늘도 예측이 +이므로 그대로 유지 (거래 없음), 예측 부호가 바뀌면 그때 거래를 실행한다. 이를 위해서는 모델 예측이 **연속성** 있게 나오도록 threshold를 걸거나 (노이즈로 인한 잦은 부호변경 방지), 혹은 **포지션 유지 비용**(공매도 비용 등)을 고려하여 결정해야 한다. 본 사례의 밈주식은 공매도 비용도 높을 수 있으므로 (논문에서 언급된 차익거래 제약 중 하나 ⁵¹), 너무 잦은 롱숏 전환은 피하는 게 좋다.

6.3 거래 비용 및 슬리피지 반영: 거래 비용은 전략 수익을 현실적으로 조정하는데 필수다. 비용에는 **브로커 수수료, 매수/매도 스프레드, 충격비용(슬리피지), 공매도 수수료** 등이 있다. 단순화를 위해 **왕복 거래비용**을 일정 비율로 뺄 수 있다. 예컨대 **0.2% (20bps)**를 한 번 포지션 전환에 드는 비용으로 가정한다 (매수/매도 각각 0.1%씩). 그러면 포지션이 바뀔 때마다 수익률에서 0.2%p를 빼 주면 된다. 포지션 유지는 비용 0.

구체적으로: - 날 t에 포지션 0 -> 날 t+1에 1 (롱 진입): 비용 0.1% 발생 (진입만 고려하거나, 진입+청산=0.2%인데 청산은 나중에). - 날 t에 1 (롱) -> 날 t+1에 -1 (숏으로 전환): 기존 롱 청산 0.1% + 신규 숏 0.1% = 0.2% 총 비용. - 날 t에 1 -> 날 t+1에 1 (유지): 비용 없음 (거래 없으므로).

이렇게 하면 매일 **실현 수익률**을 구할 때 비용을 뺄 수 있다:

$$r_{t+1}^{\text{net}} = s_t \cdot y_{t+1} - \text{Cost}_{t+1},$$

여기서 $s_t \in \{-1, 0, 1\}$ 는 t일 종가 이후 취한 포지션 (숏/중립/롱), y_{t+1} 은 t일~t+1일 실제 수익률, Cost는 위 규칙에 따른 거래비용.

Python 시뮬레이션 예시:

```
cost_rate = 0.001 # 0.1% half-turn cost
position = 0 # start neutral
strategy_returns = []
for t in range(len(test_dates)):
    # 모델 예측에 따른 신호
    signal = 1 if y_pred[t] > 0 else -1
    # 비용 계산
    if signal != position:
        # 포지션 변경 발생
        cost = cost_rate * abs(signal - position) # pos->neg: abs(-1-1)=2 -> 0.002 (0.2%)
    else:
        cost = 0.0
    # 당일 실제 수익에 따른 전략 수익
    strat_ret = signal * y_true[t] - cost
    strategy_returns.append(strat_ret)
    position = signal
```

이 루프를 돌면 `strategy_returns` 리스트에 테스트 기간 내 매일의 순수익률이 쌓인다. 이것으로 Sharpe, MDD 등을 아까 언급한 방식으로 계산한다.

6.4 전략 성능 분석: 이제 위와 같이 전략을 구성하고 나면, **평가 지표**들을 산출하여 모델별로 비교한다. 예컨대: - **누적 수익률 곡선**을 그려서 시간에 따른 성장 여부와 변동성 확인. - **연평균 수익률**과 **연평균 변동성**을 계산하여 Sharpe 산출. - **최대 낙폭 기간**을 찾아 표시 (예: 2022년 5~6월에 -15% 떨어짐). - **월별 Hit-rate** 등을 계산해 일관성 확인 (혹은 분기별 IC 등).

특히 **누적 수익률 곡선**은 눈으로 보기 쉬워 보고서에 포함하면 좋다. Matplotlib 등으로 그릴 수 있으며, 여러 모델 전략을 함께 그려 비교할 수도 있다. 다만 이 매뉴얼은 코드 실행 환경이므로, 간단한 예시로 누적곡선을 그리는 코드만 첨부한다 (실제 데이터 없으므로 의사코드 수준):

```
import matplotlib.pyplot as plt
cum_ret = (1 + np.array(strategy_returns)).cumprod()
```

```
plt.figure(figsize=(8,4))
plt.plot(test_dates, cum_ret, label="Strategy (Net)")
plt.plot(test_dates, (1 + np.array(y_buyhold)).cumprod(), label="Buy&Hold")
plt.title("Cumulative Returns of Strategy vs Buy&Hold")
plt.legend(); plt.xlabel("Date"); plt.ylabel("Cumulative Return")
plt.axhline(1.0, color='gray', linestyle='--')
```

이런 그래프는 전략이 계속 1 주변에 머물면 실패, 우상향이면 성공, 우하향이면 손실임을 직관적으로 보여준다. 논문 결과를 방해하지 않도록, **전략의 성공 여부**는 contrarian 효과를 잘 활용했는지로 판단된다. 만약 **모델 기반 전략 Sharpe**가 단순 **Buy&Hold** (해당 주식 계속 들고 있기) Sharpe보다 높고, MDD가 낮다면, 모델이 유의미한 예측력을 지닌 것으로 평가할 수 있다.

6.5 추가 고려: 실제 운용을 모사하려면 **현실적 제약**도 고려해야 한다. 예를 들어, 공매도가 불가능한 경우 숏 시그널 때는 포지션 0으로 두거나 **인버스 ETF 매수**로 대체할 수 있다. 또한 **투자 금액 규모**가 커지면 슬리피지가 커질 수 있고, **잔고 관리** (ex: 매일 포트폴리오 비중 리밸런싱)도 필요하다. 그러나 이러한 부분은 이 매뉴얼의 범위를 넘으므로, 연구 단계에서는 단순 가정하에 진행하고 **민감도 분석** 정도로 대응할 수 있다 (예: 거래비용을 0.1%, 0.2%, 0.5%로 바꿔서 Sharpe 변화를 보는 등).

정리하면, 모델 예측을 **의사결정 규칙**으로 변환하고, **거래비용**을 반영해 **전략 성과**를 측정하는 것이 이 단계의 핵심이다. 이 과정을 통해 단순한 예측 정확도를 넘어 **실제 수익화 가능성**을 평가할 수 있고, 논문의 기여를 더욱 실용적으로 어필할 수 있다.

7. 해석 가능성 및 실험 설계 (Interpretability & Experimental Design)

머신러닝 모델의 **예측 성능 향상**도 중요하지만, 금융 도메인에서는 **왜 그런 예측이 나왔는지** 이해하고 **결과를 뒷받침할 증거**를 제시하는 것이 중요하다. 이를 위해 **모델 해석 기법**과 **추가적인 실험 설계**를 활용한다. 여기서는 **SHAP 가치**, **특성 중요도**, **어블레이션(탈락) 연구**, **플라시보 테스트** 등을 다룬다.

- **SHAP (SHapley Additive exPlanations) 기반 해석:** SHAP는 각 피처가 예측에 기여한 정도를 **일종의 게임이론적 값**으로 산출해준다. 트리 모델(LightGBM, XGBoost)은 `shap.TreeExplainer`를 사용하면 비교적 쉽게 SHAP 값을 얻을 수 있다. SHAP의 이점은 **개별 예측에 대한 피처 영향도**를 알 수 있다는 것이다. 예를 들어 어떤 날 모델이 -3% 수익률을 예측했다면, SHAP 분석으로 "그날 **confidence_score**가 매우 높아서 예측을 -쪽으로 2% 움직였고, **Mentions 급증 (RS)**이 추가로 -1% 기여, 반면 **RSI 과매도** 상태라 약간 +0.5% 상향 조정" 같은 식의 **분해**가 가능하다. 이러한 결과는 논문에서 발견한 "확신 표현이 많으면 악재"라는 현상을 모델이 실제로 활용하고 있음을 증명해준다.

구현 측면에서:

```
import shap
explainer = shap.TreeExplainer(lgb_model)
shap_values = explainer.shap_values(X_val) # 검증세트에 대한 shap 값
shap.summary_plot(shap_values, X_val, feature_names=feature_cols)
```

등을 통해 **전반적 피처 영향력**을 시각화할 수 있다. `summary_plot`은 특성별 평균적인 SHAP 절대값을 보여주는데, 여기서 상위에 오는 특성이 **모델이 가장 중요하게 쓰는 요인**들이다. 만약 text 기반 특성 (예: `confidence_score`)이 price나 기술적

지표보다 상위에 있다면, **소셜미디어 요인이 모델 예측에 중요한 역할**을 하고 있음을 정량적으로 뒷받침한다. 또한 SHAP는 각 포인트 색으로 해당 특성 값 크기도 표현하므로, **특성 값 증가에 따라 예측이 +쪽으로 갔는지 -쪽으로 갔는지도** 알 수 있다. 예컨대 Reddit_mentions 특성에 대해 색이 진할수록 SHAP 값이 음수라면, **언급량이 많을수록 예측을 하락 쪽으로 움직였다는** 것이 시각적으로 드러난다.

딥러닝 모델의 경우 SHAP 계산이 까다롭지만, 가능하다면 **Deep SHAP**이나 **Integrated Gradients** 기법을 사용할 수 있다. TFT 모델은 자체적으로 **Attention 가중치**를 제공하므로, 각 시점/각 변수의 중요도를 추출해 **Heatmap**으로 나타낼 수도 있다. 예를 들어 "모델이 주목한 과거 시점"을 확인하면, 혹시 **이벤트 당일 이후 특정 LAG에서 신호를 주는지** 파악 가능하다.

- **특성 중요도 (Feature Importance):** SHAP 외에 전통적인 피쳐 중요도도 참고한다. 트리모델의 경우 **분할 이득** 기반 중요도를 제공하고, TCN/TFT는 아예 쉽게 안 나오지만, SHAP로 대체하거나, **피쳐를 하나씩 제거하며 성능 변화를 관찰**할 수 있다. 일반적으로 LightGBM의 중요도 상위권에 위치한 특성들은 (신뢰성이 완벽하지는 않지만) 모델이 많이 사용한 것들이다. 이를 정리하여, " **가장 예측력 높은 요인** 은 무엇인가"에 답할 수 있다. 예를 들어 "reddit_momentum_7이 가장 기여도가 높았으며, 그 다음이 confidence_score, RSI 순이었다"와 같이 기술하면, 향후 이 요인들에 주목한 투자전략이나 추가 연구 아이디어를 제시할 수 있다.

- **어블레이션(Ablation) 실험:** 이는 **특정 요소를 제거했을 때 성능이 어떻게 변하는지**를 보는 실험이다. 모델 측면에서는 **특정 피쳐 그룹을 제외**하고 다시 훈련하여 성능을 비교하는 방식으로 할 수 있다. 예컨대:

- 케이스 A: 모든 피쳐 사용 (Full model)
- 케이스 B: 소셜미디어 관련 피쳐 제외 (언급량, 감정 등 빼고 가격기술적 지표만 사용)
- 케이스 C: 기술적 지표 제외 (가격 관련 빼고 Reddit 텍스트만 사용)
- 케이스 D: 텍스트 확신/감정 제외 (RS나 Mentions는 포함하되 text sentiment/confidence 제외)

각 케이스별로 동일한 데이터 분할/모델링으로 테스트 성능(IC, Sharpe 등)을 구한다. 만약 케이스 B (소셜미디어 제외)의 성능이 크게 떨어진다면, **소셜미디어 요인이 예측력에 기여**한다는 방증이다. 실제로 논문 기여 중 하나가 "온라인 투자 커뮤니티의 집단 심리가 새로운 정보 제공"이었으므로, 이를 모델 실험으로 재확인하는 셈이다. 반대로 변화가 없다면, 해당 피쳐가 중복되거나 효과가 없다는 뜻이므로 모델을 간소화할 수 있다.

어블레이션 결과는 표로 정리해 제시할 수 있다:

모델 구성	IC	Hit-rate	Sharpe
Full (모든 피쳐)	0.060	0.45	1.20
- Reddit 제거	0.010	0.51	0.50
- 기술적 제거	0.040	0.43	1.05
- 텍스트 제거	0.025	0.41	0.80

(임의 숫자 예시)
위처럼 Reddit 특성을 빼면 IC가 크게 떨어졌다면, 이는 **contrarian 효과의 핵심이 Reddit 데이터에** 있음을 보이는 것이다. 기술적 지표 제거시 Sharpe가 약간 떨어지는 정도라면, 기술적 특성도 보조적 역할을 했음을 시사. 이런 식의 분석은 **정량적 기여도 파악**에 도움된다.

- **플라시보 테스트 (Placebo Tests):** 이는 **모델이 진짜 유효한 패턴을 학습한 것인지** 확인하기 위한 가짜 실험이다. 방법은 다양하지만, 몇 가지 제시하면:
- **미스매치 테스트:** Reddit 특성과 주가 수익률의 **짜를 랜덤으로 섞어** 모델을 다시 학습/평가한다. 예를 들어 날짜를 섞거나, GME의 Reddit 데이터를 AMC 주가에 붙이는 등, **논리적 인과관계가 없는 매칭**을 만들어본다. 이때 모델 성능이 무작위 수준(예: IC≈0, Sharpe≈0)으로 나와야 정상이다. 만약 랜덤 매치에서도 높은 성능이 나오면, 모델이 혹시 **시계열 자동상관** 등 다른 정보로 먹통을 맞춘 것이거나, 과적합일 수 있다. 우리의 경우, **Reddit-주가 인과성**이 없다면 성능 붕괴가 플라시보에서 보여질 것이다.

- **선행/시차 테스트:** Reddit 신호가 선행 인과라면, **반대로 해보면 성능이 안 나와야** 한다. 예컨대 Reddit 데이터를 하루 뒤로 시프트(미래 정보를 사용)해서 예측하거나, 반대로 **라벨(수익률)을 미래로 하고 과거를 맞추게** 하는 등, **타임머신 상황**을 만들어 본다. 말이 안 되는 상황에선 모델이 성능을 못 내야 한다. 이런 테스트는 "**혹시 우리의 검증에 미세한 데이터 누설이 있었나**"를 점검하는 용도로도 쓰인다.
- **무의미 특징 테스트:** 소셜미디어 외에 **무작위 잡음 특성**이나 **영동한 특성** (예: 다른 completely unrelated subreddit mention, 날씨, etc.)을 추가해보고 모델이 그것에 높은 중요도를 부여하거나 성능 향상을 보이는지 확인한다. 만약 그렇다면 모델이 **노이즈에 과적합**했을 가능성이 있다. 일반적으로 무작위 노이즈 추가는 성능 변화를 안 주거나 약간 떨어뜨려야 정상이다.
- **라벨 섞기 테스트:** 종속변수인 수익률을 **랜덤으로 섞어서** (각 날짜의 수익률을 다른 날 것으로 배치) 모델을 학습해 본다. 이때는 어떤 패턴도 없으므로, 훈련세트에서조차 성능이 평범해야 한다. 만약 훈련에서 높은 성능이 난다면 모델이 데이터를 **과대적합**하고 있음을 보여준다. 이를 통해 현재 모델의 complexity가 적절한지 점검한다.

이러한 플라시보 실험 결과도 표나 설명으로 기술한다. 예를 들어: - "랜덤 매칭 실험에서 IC는 -0.005로 유의미한 패턴이 사라졌다. 이는 본 모델이 실제 Reddit-주가 간 관계를 학습한 결과임을 시사한다." - "라벨 시프팅 실험 (미래 주가 예측 과거 언급량)은 성능이 0에 수렴하여, 우리의 검증절차에 데이터 누설이 없음을 확인하였다."

이는 연구 결과의 **강건성(robustness)**을 뒷받침하여, 독자에게 모델이 단순히 통계적 착시나 데이터 과적합이 아니라 실제 경제적 의미 있는 패턴을 포착했다는 확신을 준다.

- **추가 실험 설계:** 이 외에도 **모델 안정성 테스트** (훈련 데이터 80% 샘플로 여러 번 학습하여 성능 분포 관찰), **시간에 따른 성능 변화** (슬라이딩 윈도우 Sharpe 추이), **다른 종목에 적용** (예: 밌주식 외의 종목에 적용해서 성과 비교) 등을 생각해볼 수 있다. 예컨대 contrarian 효과가 없는 일반 주식에는 이 모델이 통하지 않아야 정상이다. 이러한 비교실험을 통해 **본 연구의 결과가 밌 주식에 특화된 것임을** 명확히 할 수 있다.

마지막으로, **해석 및 실험 결과의 조화로운 기술**이 중요하다. 즉, 앞선 본문에서 제시된 정성적 주장 (확신 표현 많으면 하락 등)과 모델 해석/실험 결과가 서로 일치해야 한다. 다행히도 논문에서는 이미 "*Overconfidence Paradox*"로 확신도와 수익률의 음의 상관을 입증했다 ⁷. 우리의 모델도 SHAP 등으로 같은 결론을 보여주었다면, "**모델 결과는 기존 분석과 일치하며, 더 나아가 이러한 요인의 상호작용까지 학습한 것으로 보인다**"고 서술할 수 있다.

또한 **한계 및 향후 연구**도 해석 파트에서 간략히 언급 가능하다. 예를 들어, "*SHAP*로 보니 모델이 주가 변동성 높을 때 예측을 잘 못하는 경향이 있었다"거나 "다른 커뮤니티(예: *Twitter*) 데이터와 비교해보는 것은 향후과제" 등. 이는 독자가 결과를 신뢰하면서도 현실적인 관점을 가지게 돕는다.

이상으로 데이터 수집부터 모델링, 전략 적용, 해석 및 검증에 이르는 전체 과정을 매뉴얼 형태로 상세히 기술하였다. 이 매뉴얼을 따라가면, 기존 연구의 기본 구조를 넘어 **예측력 향상을** 달성할 수 있는 여러 기법들을 구현하고 실험해볼 수 있을 것이다. 중요한 점은, **모든 과정에서 일관성 있는 가설 (contrarian effect)을 염두**에 두고, 그 가설을 다각도로 검증하며 모델에 녹여내는 것이다. 이를 통해 단순히 성능 좋은 모델을 넘어서, **왜 성능이 좋고 어떤 경제적 의미가 있는지** 까지 설명할 수 있는 풍부한 연구를 완성할 수 있을 것으로 기대한다.

1 2 3 4 5 6 7 8 9 10 11 12 15 28 31 32 50 51 meme stock papaer_김건희,최서빈,황유진
_182015ver 2.pdf
file:///file-Ut9XhY2sfWmZ3zC1jqmKgR

13 14 41 Temporal Fusion Transformers for Enhanced Multivariate Time Series Forecasting of Indonesian Stock Prices

<https://thesai.org/Publications/ViewPaper?Volume=15&Issue=7&Code=IJACSA&SerialNo=13>

16 22 23 42 43 44 45 Advances in Financial Machine Learning · Reasonable Deviations

https://reasonabledeviations.com/notes/adv_fin_ml/

17 18 How To Backtest Machine Learning Models for Time Series Forecasting - MachineLearningMastery.com

<https://machinelearningmastery.com/backtest-machine-learning-models-time-series-forecasting/>

19 20 Purged K-Fold CV in Financial Time-Series | Medium

<https://medium.com/@saveriomazza/why-k-fold-cross-validation-fails-in-finance-and-how-to-fix-it-a-deep-dive-de6e00b913b6>

21 24 The 7 Reasons Most Machine Learning Funds Fail - Medium

<https://medium.com/the-quant-journey/the-7-reasons-most-machine-learning-funds-fail-a1faec389a6f>

25 Walk forward validation : r/datascience - Reddit

https://www.reddit.com/r/datascience/comments/18pxc6x/walk_forward_validation/

26 How To Backtest Machine Learning Models for Time Series ...

<https://www.machinelearningmastery.com/backtest-machine-learning-models-time-series-forecasting/>

27 46 47 48 49 Learning Universal Multi-level Market Irrationality Factors to Improve Stock Return Forecasting

<https://arxiv.org/html/2502.04737v1>

29 30 35 36 37 38 Temporal Convolutional Network — An Overview | by Amit Yadav | Medium

<https://medium.com/@amit25173/temporal-convolutional-network-an-overview-4d2b6f03d6f8>

33 34 Temporal Convolutional Networks and Forecasting - Unit8

<https://unit8.com/resources/temporal-convolutional-networks-and-forecasting/>

39 Multi-Sensor Temporal Fusion Transformer for Stock Performance ...

<https://www.mdpi.com/1424-8220/25/3/976>

40 Demand forecasting with the Temporal Fusion Transformer

<https://pytorch-forecasting.readthedocs.io/en/v1.4.0/tutorials/stallion.html>